

DASE7506 MP1: Small Language Model Challenge

Student Name: Zhuo ZHENG
Student ID: 3036780228

September 2026

Abstract

This report presents a from-scratch decoder-only Transformer for the fixed WikiText-2 byte-level language-modeling challenge. The method builds on the supplied GPT by increasing representational capacity, replacing learned absolute positions with rotary position embeddings (RoPE), applying dropout, and mixing the language-model distribution with a causal within-window neural cache. Stochastic weight averaging (SWA) selects a smoother late-training predictor. At the same 10,000-step and 81.92-million-target budget as the baseline, the selected system reduces test bits per byte (BPB) from 1.7125 to 1.5410. It uses 4.84 times as many parameters and 4.12 times the measured CPU scoring time while meeting the memory, asset, and timing limits. The discussion focuses on what each component is intended to do and what the controlled comparisons can establish. Dropout shows the largest ablation effect; the cache provides a smaller gain. Since the experiments use one primary seed, component-level conclusions remain specific to this training run.

1 Problem Setting and Evaluation

The task is to minimize full-test BPB on supplied WikiText-2 raw text using the fixed training-fitted BPE-2048 tokenizer. The FP32 evaluator scores next-token predictions in independent 256-token windows and divides total negative log likelihood in bits by the split’s UTF-8 byte count. Lower BPB is better; it is not token perplexity.

The limits are 5x baseline CPU time, 4 GiB peak RAM, and 64 MiB of uncompressed inference assets. The method uses no pretrained weights or external text, and keeps predictions causal with no state carried between windows. Validation BPB selects the checkpoint; the frozen test score is reported only after selection.

2 The Supplied Baseline

2.1 Architecture and training behavior

The reference model is a small GPT-style decoder-only Transformer using the core self-attention, residual, and feed-forward components introduced by Vaswani et al. [1]. It embeds each token and adds a learned absolute position embedding for positions 0 through 255. Its four pre-normalized blocks each contain causal self-attention followed by a two-layer GELU feed-forward network with hidden width four times the model width. Each sublayer has a residual connection. A final

LayerNorm feeds a vocabulary projection. The token embedding and output projection share weights.

The baseline has vocabulary 2,048, width 128, 4 heads, depth 4, and context 256 (1,088,256 parameters). It uses learned absolute positions and causal scaled dot-product attention, with no dropout, cache, or checkpoint averaging. Its simple architecture provides a clear reference for testing capacity and positional encoding.

2.2 Baseline score under the matched training budget

For a fair comparison, the baseline was retrained for 10,000 steps with seed 17, batch size 32, and 256 targets per sequence: 81.92 million targets, matching the final model. It scored 1.689167 validation BPB and 1.712524 test BPB, with 10.76-second CPU scoring time. Baseline training took 210 seconds. The comparison controls data, tokenizer, evaluator, and target count, but not capacity or components; it measures the complete system gain.

3 Proposed Model

3.1 Architecture and positional encoding

The final network has width 256, 8 heads, 6 sequential decoder blocks, and context length 256. Each block retains the baseline’s pre-LayerNorm, causal attention, and GELU MLP with hidden size $4d$. The final model uses dropout probability 0.1 at the embedding, attention, and residual-output paths. The token embedding and output matrix remain tied. The configuration contains 5,263,362 trainable parameters.

The principal positional change replaces the learned absolute table with RoPE [2]. Within each attention head, pairs of query and key coordinates are rotated by a phase determined by token position. Their dot products then depend on relative displacement as well as token content. RoPE does not add a learned position table and leaves the causal attention mask unchanged. In this project the context remains fixed at 256; the motivation is a useful relative-position inductive bias, not an untested claim of extrapolation to longer sequences.

The final model retains sequential attention-then-MLP residual blocks, LayerNorm, and GELU. These stable baseline components keep the implementation simple; the final model does not include parallel blocks, QK normalization, or SwiGLU.

3.2 Causal neural cache

The neural cache adds a local next-token distribution based on hidden features from the already observed prefix of the current window. Let h_t denote the final normalized feature at position t . For each earlier position $j < t$, it computes a scaled cosine similarity, masks all positions $j \geq t$, and

normalizes the remaining scores:

$$s_{tj} = \alpha \frac{h_t^\top h_j}{\|h_t\| \|h_j\|}, \quad (1)$$

$$a_{tj} = \text{softmax}_{j < t}(s_{tj}), \quad (2)$$

$$c_t(v) = \sum_{j < t} a_{tj} \mathbf{1}[x_{j+1} = v]. \quad (3)$$

Each feature h_j is associated with its observed successor x_{j+1} . At position t , only $j < t$ is allowed, so each voted-for successor is already observed; $j = t$ is excluded because its successor is the target. The cache gate is zero at the first position.

The scale α is learned and clamped to $[1, 100]$. The cache distribution is mixed with the ordinary model distribution using a learned scalar gate $\gamma = \sigma(g)$:

$$p_t(v) = (1 - \gamma)p_{\text{LM},t}(v) + \gamma c_t(v).$$

The cache is differentiable and trained jointly with the decoder, similar in spirit to continuous-cache language modeling [3]. It uses no external corpus or stored answers: temporary state is recomputed per window. Similarity computation is quadratic in context length, so the 256-token limit is central to its measured cost.

3.3 Optimization and checkpoint selection

The final run uses seed 17, 10,000 updates, batch size 32, and 256 targets per sequence. It uses AdamW (learning rate 10^{-3} , weight decay 0.1), 100-step linear warm-up, cosine decay, gradient clipping at 1.0, and bf16 training; scoring is FP32. Following stochastic weight averaging (SWA) [4], the checkpoint averages each update from step 6,000 to 10,000. Validation selects the average (1.517091 BPB) over the raw final checkpoint (1.522602). Training ran on an NVIDIA GeForce RTX 3050 Ti Laptop GPU and took 1,095 seconds (18.3 minutes); the matched baseline took 210 seconds (3.5 minutes) on the same GPU.

4 Method Design and Expected Effects

The components address different limitations: capacity improves representation, RoPE encodes relative position, the cache reuses local continuations, dropout regularizes training, and SWA smooths late-stage weights. The table summarizes their intended roles and the evidence available for this run.

4.1 Capacity: widening and adding depth

Width, heads, and depth increase from 128/4/4 to 256/8/6. This provides more feature channels, attention subspaces, and sequential transformations, but also raises overfitting and inference-cost risks. The short-run 256/8 comparison improved validation BPB, though width and head count changed together. The final model has 4.84x the parameters; matched target count does not mean matched FLOPs or training time.

4.2 Position representation: why use RoPE

Self-attention compares query and key vectors, but the model still needs information about token order. The baseline adds a learned vector for each absolute position. RoPE instead applies rotations to query/key coordinate pairs. As a result, the query-key interaction carries information about relative displacement. This is a useful inductive bias for language, where the relation between two tokens often matters more than their absolute index within a window. RoPE also avoids learning an independent position embedding table and integrates directly into attention.

The context remains 256, so long-context extrapolation is not tested. The attached ablation runs do not isolate RoPE; therefore this report makes no component-level score claim for its effect. Its role here is the positional inductive bias, not an independently demonstrated source of the final gain.

4.3 Local reuse: what the cache contributes

The cache complements the softmax head: similar earlier hidden states vote for their observed successor, while a learned gate controls the mixture. This can reuse repeated local continuations without an external retrieval store. Its validation gain is 0.006904 BPB, small relative to the total improvement; the quadratic similarity operation is its main cost. Longer contexts require new timing measurements.

4.4 Regularization and parameter averaging

Dropout suppresses activations during training to reduce co-adaptation; it is disabled at inference. Removing it worsened validation BPB by 0.177748, the largest measured ablation effect. SWA averages late parameter values without extra inference passes: it improved over the raw terminal checkpoint by 0.005511 BPB. EMA weight averaging [5] was only 0.000856 worse than SWA, so the evidence favors averaging but not one averaging method decisively.

4.5 Optimization choices and model selection

The selected checkpoint is chosen on validation BPB before test scoring. Alternative normalizations, SwiGLU, and parallel residual blocks were not retained because nearby tests showed no consistent benefit; this is not evidence that they are generally inferior.

4.6 Component interactions and causal boundaries

The components are trained together, so their effects need not add independently. The cache compares the final hidden features, which are themselves shaped by the decoder, positional encoding, and dropout. RoPE may help the decoder represent relative context, while the cache may exploit repeated representations; however, no experiment isolates this interaction. The cache’s cosine normalization reduces sensitivity to feature magnitude, and its learned scale and mixture gate allow training to adjust how sharp and influential retrieval should be. If local matches are unreliable, a small gate can limit their effect. The single cache ablation measures the aggregate difference with and without this mechanism, not which cache parameter is responsible.

Table 1: Component roles, measured validation evidence, and main trade-offs.

Component	Intended role	Evidence in supplied runs	Main cost / caveat
Capacity	Richer features and deeper transformations	Final system has 5.26M parameters	4.84x baseline parameters
RoPE	Encode relative token offsets in attention	Not isolated in supplied ablations	Context fixed at 256
Dropout	Regularize the higher-capacity model	Ablation BPB increases by 0.177748	Training-only stochasticity
Neural cache	Reuse local observed continuations	Ablation BPB increases by 0.006904	Quadratic context cost
SWA	Smooth late-training parameters	Improves BPB by 0.005511 vs. raw	EMA is similar

The causality argument depends on both feature construction and successor indexing. The Transformer feature h_j is causal and therefore depends only on tokens through x_j . At prediction position t , the cache includes only $j < t$. Its associated successor x_{j+1} is thus no later than x_t , which is already observed when predicting the next token. Excluding $j = t$ is essential: including it would expose the target through its successor label. The cache is reconstructed for every 256-token window, so even a token that appeared in a previous window cannot influence the current prediction through hidden state. These rules preserve the scorer contract while still allowing reuse inside a window.

Dropout and cache also meet at the representation. During training, dropout regularizes activations that contribute to hidden features; at scoring, dropout is disabled and features are deterministic. This train/evaluation difference is intentional, but it means the cache operates on a deterministic feature geometry at test time that was learned under activation noise. The strong no-dropout regression suggests that this regularization is useful overall, but the current experiments do not separate its direct effect on the language-model head from its indirect effect on cache similarity.

SWA changes parameters rather than predictions. It therefore adds no inference ensemble and does not multiply the number of forward passes. Its benefit depends on late checkpoints lying in a region where parameter averaging remains useful; the validation comparison supports it for this trajectory.

5 Final Results

5.1 Matched-target comparison

Table 2 gives the principal benchmark comparison. Both models processed 81.92 million training targets and use the same fixed scorer, tokenizer, data, and FP32 evaluation protocol. The final model reduces validation BPB by 0.172075 and test BPB by 0.171557, approximately a 10.0% reduction relative to the baseline test score. Since model capacity and components differ, this comparison measures the combined system improvement, not the isolated effect of RoPE or the cache.

The final model has 4.84x the baseline parameters and scores in 4.12x the CPU time, below the 5x cap. Peak RSS is 2.085 GiB and inference assets are 21 MiB, both within budget.

Table 2: Matched-target comparison with the supplied baseline. Lower BPB is better.

Metric	Baseline	Final
Parameters	1,088,256	5,263,362
Training steps	10,000	10,000
Processed training targets	81.92M	81.92M
Validation BPB	1.689167	1.517091
Test BPB (FP32)	1.712524	1.540968
CPU scoring time	10.76 s	44.34 s
Peak evaluation RSS	Not recorded	2.085 GiB
Uncompressed inference assets	Not recorded	21 MiB

6 Ablation Studies

All ablations below use the attached 10,000-step runs and the final checkpoint. They share seed 17, batch size 32, 81.92 million training targets, and the fixed evaluator. Validation BPB is the model-selection metric; test BPB is included as the archived evaluation of each frozen checkpoint, not as a tuning criterion. The comparisons are single-seed.

6.1 Module 1: Parameter averaging

This comparison holds the model and training run fixed and changes only how weights are selected: raw terminal checkpoint, EMA from step 6,000, or SWA from step 6,000. Both averaging methods improve over the raw checkpoint. SWA is slightly better than EMA, but their small difference does not support a strong ranking.

Table 3: Averaging ablation. Lower BPB is better; deltas are relative to the raw checkpoint.

Checkpoint	Validation BPB	Test BPB	Δ Val.	Δ Test
No averaging	1.522602	1.547216	–	–
EMA, start 6,000	1.517947	1.541668	-0.004655	-0.005549
SWA, start 6,000 (final)	1.517091	1.540968	-0.005511	-0.006249

The validation and test results agree on the direction: averaging improves the raw checkpoint. EMA and SWA differ by only 0.000856 validation BPB and 0.000700 test BPB. This suggests that the measurable benefit is primarily from averaging late checkpoints rather than a decisive advantage of one averaging rule.

6.2 Module 2: Dropout

This ablation compares dropout 0.1 with dropout disabled, keeping the cache and SWA enabled. Removing dropout raises validation BPB by 0.177748 and test BPB by 0.182664, the largest effect among the attached component comparisons.

The large degradation is consistent with dropout regularizing this higher-capacity model on the supplied corpus.

Table 4: Dropout ablation against the final SWA checkpoint.

Configuration	Validation BPB	Test BPB	Δ Val.	Δ Test
Dropout 0.1 (final)	1.517091	1.540968	–	–
Dropout 0.0	1.694840	1.723632	+0.177748	+0.182664

6.3 Module 3: Neural cache

The cache ablation changes only the cache switch while retaining dropout 0.1 and SWA. Removing it raises validation BPB by 0.006904 and test BPB by 0.004071.

Table 5: Cache ablation against the final SWA checkpoint.

Configuration	Validation BPB	Test BPB	Δ Val.	Δ Test
Cache enabled (final)	1.517091	1.540968	–	–
Cache disabled	1.523996	1.545038	+0.006904	+0.004071

The cache gives a modest, directionally consistent improvement in both splits. Its effect is much smaller than dropout’s, so it should be viewed as a useful addition within this context-length and CPU budget, not as the main source of the overall gain.

7 Discussion and Limitations

The principal trade-off is predictive quality versus CPU cost: the final model uses 4.12x scoring time, close to the 5x limit. Cache adds modest quality at quadratic context cost; dropout has a larger measured validation effect. Evidence is limited to one corpus, one tokenizer/window protocol, and one primary seed. The final and baseline training runs took 1,095 and 210 seconds respectively; aggregate search time is unavailable. Replicated seeds and longer-context timing tests are the clearest follow-ups.

The paired baseline comparison is strong evidence for the complete system because training targets and evaluation conditions match. The supplied ablations isolate averaging, cache, and dropout, but do not isolate width, depth, or RoPE. The cache’s 0.0069-BPB validation effect could be sensitive to initialization or validation sampling; the larger dropout effect is more compelling but still lacks a seed-based uncertainty estimate.

Repeated checkpoint and hyperparameter selection on one validation split can also overfit development decisions to that split, even without using test data. The similar final validation and test improvements reduce concern about a large validation-only gain, but one test evaluation cannot estimate generalization variance. A stronger follow-up would repeat the final and ablation configurations under several seeds, report mean and spread, and keep the same frozen test protocol. A per-window or per-category error analysis could then show whether the cache helps repeated local patterns specifically or whether gains are distributed across the corpus.

The inference budget is measured for the frozen predictor, whereas the broader search cost is not fully summarized. The recorded final and baseline training times are 1,095 and 210 seconds, but the aggregate time spent testing alternative settings is unavailable. Consequently, the report can compare inference efficiency and the principal training runs, but cannot claim that the search itself

was compute-efficient. This limitation does not alter the benchmark score, but it matters when judging the method’s total development cost.

8 Conclusion

At an equal 81.92-million-target budget, the final model reduces test BPB from 1.712524 to 1.540968 and meets the inference limits. Dropout shows the largest ablation effect; the cache provides a smaller gain. The overall improvement is supported, but single-seed evidence limits attribution to individual components.

9 Reproducibility, Data, and AI Disclosure

Installation, training, and evaluation commands are documented in the repository `README.md`; this report focuses on the model and experimental evidence. The archived final checkpoint is `code/final/full-10000/swa-checkpoint.pt`, evaluated with protocol `7506-mp1-wt2-v2`. The supplied evaluator and tokenizer remain unchanged.

WikiText-2 was introduced by Merity et al. [6]. The text is by Wikipedia contributors. The [upstream dataset page](#) identifies the dataset under the [Creative Commons Attribution-ShareAlike 3.0 License](#) and the [GNU Free Documentation License](#).

GitHub Copilot assisted with implementation and modification of parts of the code, and provided writing and editing support during report preparation.

References

- [1] A. Vaswani et al., “Attention Is All You Need,” in *Advances in Neural Information Processing Systems*, 2017.
- [2] J. Su et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” arXiv:2104.09864, 2021.
- [3] E. Grave, A. Joulin, and N. Usunier, “Improving Neural Language Models with a Continuous Cache,” arXiv:1612.04426, 2016.
- [4] P. Izmailov, D. Podoprikin, T. Garipov, D. Vetrov, and A. G. Wilson, “Averaging Weights Leads to Wider Optima and Better Generalization,” arXiv:1803.05407, 2018.
- [5] A. Tarvainen and H. Valpola, “Mean Teachers Are Better Role Models: Weight-Averaged Consistency Targets Improve Semi-Supervised Deep Learning Results,” arXiv:1703.01780, 2017.
- [6] S. Merity, C. Xiong, J. Bradbury, and R. Socher, “Pointer Sentinel Mixture Models,” arXiv:1609.07843, 2016.